

Parte 1:

En la clase Object:

```
public native int hashCode(); //Retorna un valor hashcode para un objeto.
```

Para Integer: utiliza el valor del entero

@Override

```
public int hashCode() {  
    return value;  
}
```

Para String: utiliza esta fórmula basada en el valor ASCII de los caracteres y el numero primo 31.

@Override

```
public int hashCode() {  
    int h = 0;  
    for (int i = 0; i < value.length; i++) {  
        h = 31 * h + value[i];  
    }  
    return h;  
}
```

La diferencia se debe a que cada clase tiene distintas necesidades semánticas:

1. Object no conoce los datos internos del objeto, solo puede garantizar identidad.
2. Integer representa un único valor numérico, tiene sentido que el hash sea el mismo número y facilita comparaciones rápidas y coherencia con equals().
3. String es una secuencia de caracteres, necesita que cadenas distintas tengan hashes distintos en lo posible, busca distribuir uniformemente los valores en tablas hash y usa una función matemática para minimizar colisiones.

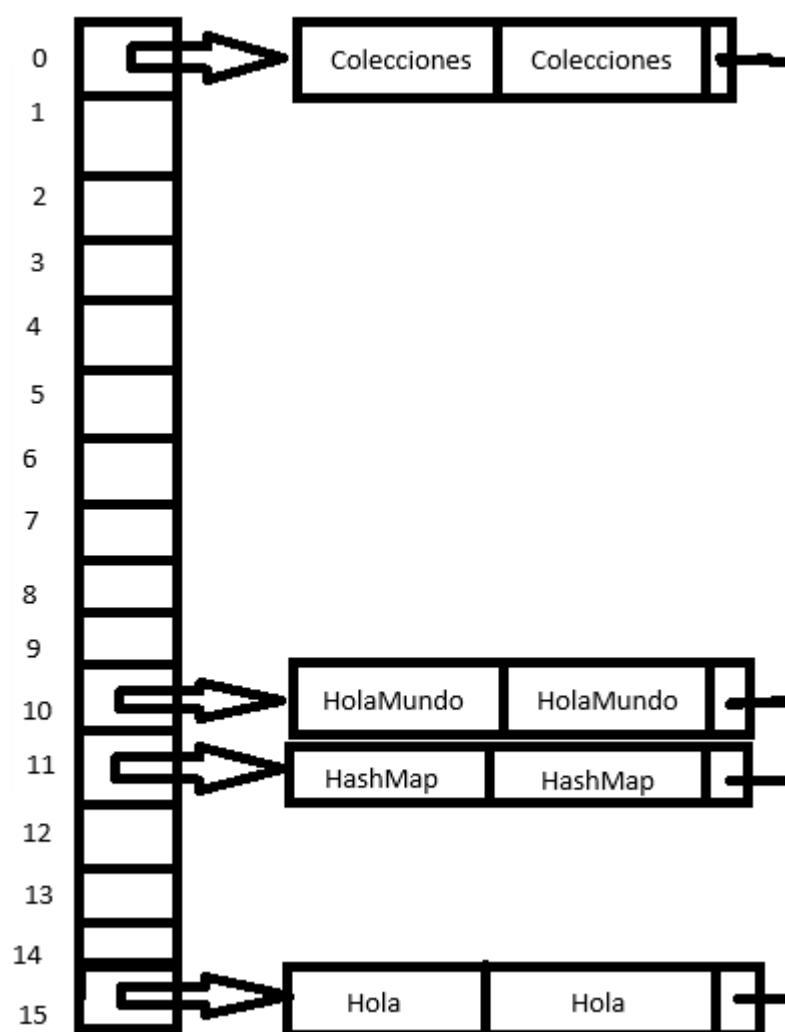
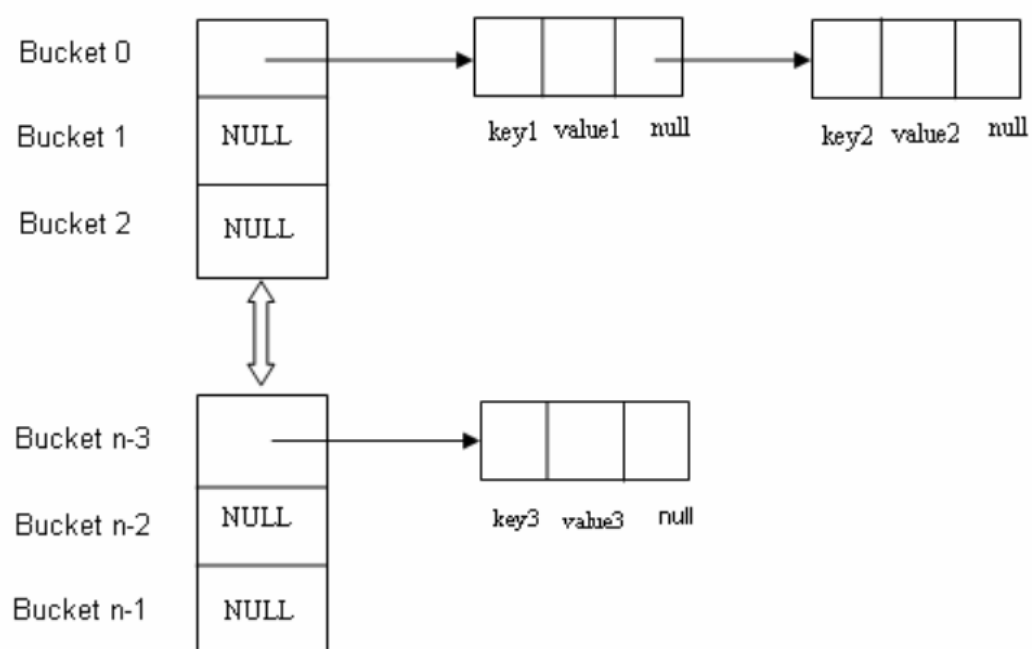
Parte 2:

Un HashMap es una estructura de datos que almacena pares clave → valor y permite acceder a ellos muy rápidamente usando una función hash

Internamente, un HashMap combina:

Un arreglo (array) de posiciones llamadas buckets.

Listas enlazadas o árboles dentro de cada bucket para manejar colisiones.



Parte 3:

Equals y hashCode estan implementadas en codigo. Caracteristicas:

Si dos objetos son iguales segun equals() es porque tienen el mismo hashCode, pero si dos objetos tienen el mismo hashCode no necesariamente son iguales.

Que sean consistentes. Los objetos siempre deben devolver el mismo valor.

Y que usen los mismos atributos. Si uso id en equals, uso id en hashCode.